

Техническое задание на разработку высокопроизводительной системы отслеживания ошибок «BugZero»

1. Введение: Концепция и философия продукта

1.1. Эволюция систем багтрекинга и определение целей

В современной экосистеме разработки программного обеспечения наблюдается фундаментальный сдвиг в требованиях к инструментам управления задачами. Традиционные системы, такие как Jira или Bugzilla, которые долгое время доминировали на рынке, часто подвергаются критике за избыточную сложность, перегруженность интерфейса и низкую производительность, что приводит к увеличению когнитивной нагрузки на разработчиков.¹ Запрос на «простую, удобную и функциональную» систему подразумевает не просто уменьшение количества кнопок, а смену парадигмы: переход от инструмента учета ошибок к инструменту *повышения продуктивности*.

Проектируемая система «BugZero» ставит своей целью реализацию концепции «Issue Tracking as a Productivity Tool». В отличие от универсальных комбайнов, которые пытаются удовлетворить потребности HR, маркетинга и бухгалтерии одновременно, BugZero фокусируется исключительно на инженерных командах. Ключевые принципы, заложенные в основу проектирования, базируются на анализе лучших практик таких инструментов, как Linear и современных стандартов UX.¹

Ключевые ценности системы:

1. **Скорость как функциональность (Performance as a Feature):** Система должна обеспечивать мгновенный отклик интерфейса (менее 100 мс) на действия пользователя. Это достигается за счет использования оптимистичных обновлений (Optimistic UI) и локального кэширования данных, минимизируя время ожидания ответа от сервера.³
2. **Минимализм и контекстность:** Интерфейс не должен требовать настройки сотен полей для создания простой задачи. Данные должны запрашиваться только тогда, когда они действительно необходимы для текущего этапа жизненного цикла бага.⁵
3. **Клавиатуроцентричность (Keyboard-First Design):** Профессиональные пользователи (разработчики) должны иметь возможность выполнять 90% операций без использования мыши, используя командную строку (Command Palette) и горячие клавиши, что существенно ускоряет навигацию и выполнение рутинных действий.²
4. **Гибкость методологий:** Поддержка гибридных процессов, сочетающих элементы Kanban (для непрерывного потока задач) и Scrum (для итеративной разработки)

спринтами), без навязывания жестких ограничений.⁷

1.2. Анализ целевой аудитории и сценариев использования

Понимание конечного пользователя является критическим фактором для принятия архитектурных решений. Мы ориентируемся на кросс-функциональные команды разработки программного обеспечения размером от 5 до 50 человек.

- **Разработчики (Software Engineers):** Для них критична интеграция с кодовой базой (GitHub/GitLab), возможность быстро привязать задачу к ветке или пулл-реквесту, а также минимальное переключение контекста. Исследования показывают, что переключение контекста может снижать продуктивность на 20-30%.⁹ BugZero должна минимизировать это время, предоставляя быстрый доступ к задачам прямо из среды разработки или через быстрые команды.
- **QA-инженеры (Quality Assurance):** Основная потребность — структурированность отчетов об ошибках. Система должна предоставлять шаблоны, требующие обязательного заполнения шагов воспроизведения, ожидаемого и фактического результата, а также поддерживать удобную загрузку медиа-файлов (скриншотов, логов).¹⁰
- **Продакт-менеджеры (Product Managers):** Им необходима прозрачность процессов: понимание того, на какой стадии находится разработка, автоматический расчет скорости команды (Velocity) и визуализация прогресса (Burn-down charts). Важна возможность приоритизации бэклога и управления дорожной картой.⁷

2. Сравнительный анализ аналогов и обоснование кастомной разработки

Перед началом проектирования необходимо проанализировать существующие решения, чтобы выявить их слабые стороны и интегрировать лучшие практики в BugZero.

2.1. Анализ конкурентных решений

Система	Сильные стороны	Слабые стороны	Уроки для BugZero
Jira	Невероятная гибкость, мощная отчетность, огромная экосистема плагинов. ⁵	Медленный интерфейс, сложность настройки (требуется администратор),	Отказ от сложной конфигурации workflows в пользу разумных дефолтов. Приоритет

		перегруженный UI, высокая стоимость для небольших команд.	скорости UI.
Linear	Эталон скорости, "Local-first" архитектура, отличный UX для разработчиков, встроенная работа с циклами. ¹	Ограниченная кастомизация, отсутствие сложных Enterprise-функций и отчетности.	Ориентир для UI/UX. Использование концепции "Cycles" вместо жестких спринтов.
Trello	Максимальная визуальная простота, низкий порог входа. ⁵	Не подходит для сложного багтрекинга (нет типизации полей, связей задач), сложно масштабируется.	Использование Kanban-доски как одного из представлений, но с сохранением реляционной структуры данных.
GitHub Issues	Бесшовная интеграция с кодом, бесплатно для открытых проектов. ²	Ограниченные возможности проектного управления (отсутствие диаграмм Ганта, сложных зависимостей), примитивный workflow.	Необходимость глубокой интеграции с Git, но с более мощным слоем управления проектами.

2.2. Обоснование собственной разработки

Хотя рынок насыщен решениями, создание собственной системы BugZero оправдано следующими факторами:

1. **Контроль над данными и безопасностью:** Возможность развертывания on-premise или в изолированном облаке, что критично для компаний с жесткими требованиями безопасности (например, FinTech), где данные о багах могут содержать чувствительную информацию.¹³
2. **Специфическая оптимизация процессов:** Возможность реализовать уникальные

для компании метрики и отчеты (SLA, MTTR), которые сложно настроить в коробочных продуктах без дорогостоящих плагинов.¹⁴

3. **Отсутствие Vendor Lock-in:** Использование открытого стека технологий (PostgreSQL, Next.js) гарантирует, что данные всегда будут доступны и переносимы, в отличие от проприетарных форматов Jira или Linear.¹⁵

3. Архитектура системы и технологический стек

Выбор технологического стека для BugZero обусловлен требованием высокой производительности, масштабируемости и удобства разработки (Developer Experience). Мы будем использовать современные, но проверенные технологии.

3.1. Frontend и Application Framework: Next.js 15

Для реализации клиентской части и серверной логики приложения выбран фреймворк **Next.js 15** (App Router).

- **Server Components (RSC):** Использование серверных компонентов React позволяет перенести логику рендеринга тяжелых частей интерфейса (например, больших списков задач или дашбордов аналитики) на сервер. Это существенно снижает размер JavaScript-бандла, отправляемого на клиент, ускоряя первую отрисовку (FCP) и TTI.¹⁶
- **Server Actions:** Это ключевая технология для реализации мутаций данных (создание, обновление, удаление задач). Server Actions позволяют вызывать серверные функции прямо из клиентских компонентов, обеспечивая полную типобезопасность (End-to-End Type Safety) и устраняя необходимость в создании отдельного REST API слоя для внутренних нужд приложения.¹⁷
- **Сравнение с альтернативами:** В отличие от Remix, который также предлагает отличные паттерны работы с данными, Next.js обладает более широкой экосистемой и лучшей поддержкой Vercel для деплоя на Edge. Vue.js, хотя и является отличным фреймворком, в связке с Nuxt требует больше усилий для достижения аналогичного уровня интеграции с серверной частью, как это реализовано в Next.js 15.¹⁹

3.2. База данных и Backend-as-a-Service: Supabase (PostgreSQL)

В качестве основного хранилища данных и платформы для бэкенд-сервисов выбрана **Supabase**.

- **Реляционная природа (PostgreSQL):** Багтрекер — это система с высокой степенью связности данных (Задачи связаны с Пользователями, Проектами, Комментариями, Вложениями, Тегами). Реляционная модель PostgreSQL идеально подходит для обеспечения целостности этих связей, в отличие от NoSQL решений типа Firebase, где моделирование таких связей приводит к дублированию данных и сложности поддержки.²¹

- **Realtime Subscriptions:** Supabase предоставляет функциональность Realtime "из коробки". Клиент может подписаться на изменения в конкретных таблицах (например, issues), получая уведомления по WebSocket при любом изменении данных (INSERT, UPDATE, DELETE). Это позволяет реализовать мгновенное обновление Kanban-доски у всех участников команды без необходимости писать собственный WebSocket-сервер или использовать polling.²³
- **Аутентификация и Безопасность (RLS):** Supabase предоставляет готовую систему аутентификации и механизм Row Level Security (RLS), который позволяет перенести логику разграничения доступа непосредственно на уровень базы данных. Это критически важно для безопасности: даже если API будет скомпрометировано, злоумышленник не сможет получить данные, к которым у его пользователя нет доступа на уровне БД.²⁵

3.3. ORM и управление схемой: Prisma

Для взаимодействия с базой данных используется **Prisma ORM**.

- **Декларативная схема:** Описание схемы базы данных в файле schema.prisma служит единственным источником правды для структуры данных.
- **Типобезопасность:** Prisma генерирует TypeScript-клиент на основе схемы, что гарантирует соответствие типов в коде приложения реальной структуре базы данных. Это предотвращает целый класс ошибок времени выполнения.²⁷
- **Миграции:** Prisma Migrate обеспечивает надежный механизм версионирования схемы базы данных и применения изменений, что необходимо для командной разработки и CI/CD процессов.²⁹

3.4. Архитектура синхронизации (Local-First элементы)

Хотя полная реализация Local-First (как в Linear с использованием собственных sync-движков) является технически сложной задачей, BugZero будет использовать гибридный подход для достижения ощущения мгновенной работы:

- **Optimistic UI:** Все мутации (создание задачи, смена статуса) применяются к локальному стейту интерфейса немедленно, до получения ответа от сервера. В случае ошибки происходит автоматический откат (rollback). Это создает ощущение нулевой задержки.³⁰
- **Кэширование:** Использование TanStack Query (React Query) для управления серверным состоянием на клиенте. Данные кэшируются в памяти браузера, что позволяет мгновенно отображать ранее загруженные страницы при навигации "назад-вперед" без повторных запросов к серверу.

4. Проектирование базы данных и схема данных

Эффективная структура базы данных — фундамент производительности и расширяемости системы. Схема разработана с учетом требований к поддержке

спринтов, Kanban-досок и детального трекинга.

4.1. Концептуальная модель (ER Diagram Narrative)

Система строится вокруг центральной сущности Issue (Задача).

- **Организация:** Верхнеуровневая сущность Workspace или Organization (для SaaS мульти-тенанси). Внутри организации существуют Team (Команды) и Project (Проекты).
- **Пользователи:** User связаны с командами через таблицу TeamMember, определяющую их роли.
- **Процесс:** Issue принадлежит команде и может быть привязана к Cycle (Спринту) и Project. У задачи есть Status (из настраиваемого воркфлоу команды) и Priority.
- **Метаданные:** Задачи могут иметь множество Label (Тегов), Comment (Комментариев) и Attachment (Вложений).

4.2. Детальная схема Prisma

Ниже приведено детальное описание схемы данных с учетом всех требований.²⁷

Code snippet

```
// schema.prisma
```

```
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
  directUrl = env("DIRECT_URL")  
}
```

```
generator client {  
  provider = "prisma-client-js"  
}
```

```
enum UserRole {  
  ADMIN  
  MEMBER  
  GUEST  
}
```

```
enum Priority {
```

```
NONE
LOW
MEDIUM
HIGH
URGENT
}
```

```
enum IssueStatusType {
    BACKLOG
    TODO
    IN_PROGRESS
    DONE
    CANCELED
}
```

```
// Пользователь системы
```

```
model User {
    id      String  @id @default(uuid())
    email   String  @unique
    name    String?
    avatarUrl String?

    // Связи
    createdIssues Issue @relation("Reporter")
    assignedIssues Issue @relation("Assignee")
    comments      Comment
    teams         TeamMember

    createdAt DateTime @default(now())
    updatedAt  DateTime @updatedAt

    @@map("users")
}
```

```
// Команда (например, "Backend Team", "Mobile Team")
```

```
model Team {
    id      String  @id @default(cuid())
    key     String   @unique // Префикс для задач (напр. "BACK")
    name    String
    description String?

    members TeamMember
    issues  Issue
}
```

```
cycles    Cycle
labels    Label
statuses  WorkflowStatus
```

```
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
```

```
@@map("teams")
}
```

```
// Связь многие-ко-многим между пользователями и командами с ролями
```

```
model TeamMember {
  id    String @id @default(cuid())
  userId String
  teamId String
  role  UserRole @default(MEMBER)
```

```
  user  User @relation(fields: [userId], references: [id], onDelete: Cascade)
  team  Team @relation(fields: [teamId], references: [id], onDelete: Cascade)
```

```
  @@unique([userId, teamId])
  @@map("team_members")
}
```

```
// Настраиваемые статусы для каждой команды
```

```
model WorkflowStatus {
  id      String @id @default(cuid())
  name    String // Название статуса (напр. "Code Review")
  type    IssueStatusType // Категория статуса (для группировки)
  position Int // Порядок сортировки на доске
  teamId  String
  team    Team @relation(fields: [teamId], references: [id], onDelete: Cascade)
  issues  Issue
```

```
  @@map("workflow_statuses")
}
```

```
// Задача
```

```
model Issue {
  id      Int @id @default(autoincrement()) // Внутренний числовой ID
  readableId String @unique // Человекочитаемый ID (напр. "BACK-123")
  title   String
  description String? @db.Text // Markdown контент
```



```

priority Priority @default(NONE)

// Статус задачи (ссылка на WorkflowStatus)
statusId String
status WorkflowStatus @relation(fields: [statusId], references: [id])

// Привязки
teamId String
team Team @relation(fields: [teamId], references: [id])

reporterId String
reporter User @relation("Reporter", fields: [reporterId], references: [id])

assigneeId String?
assignee User? @relation("Assignee", fields: [assigneeId], references: [id])

cycleId String?
cycle Cycle? @relation(fields: [cycleId], references: [id])

projectId String?
project Project? @relation(fields: [projectId], references: [id])

parentId Int?
parent Issue? @relation("ParentChild", fields: [parentId], references: [id])
children Issue @relation("ParentChild")

comments Comment
labels LabelOnIssue
history IssueHistory

createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

// Индексы для оптимизации выборки
@@index([teamId, statusId]) // Для Kanban доски
@@index([assigneeId]) // Для фильтра "Мои задачи"
@@index([cycleId]) // Для просмотра спринта
@@map("issues")
}

// Проект (группа задач с общей целью)
model Project {
  id String @id @default(cuid())

```

```

    name    String
    description String?
    startDate DateTime?
    targetDate DateTime?

    issues    Issue

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    @@map("projects")
}

// Цикл / Спринт
model Cycle {
    id    String @id @default(cuid())
    name    String // Напр. "Sprint 24"
    startsAt DateTime
    endsAt    DateTime
    teamId    String
    team    Team @relation(fields: [teamId], references: [id], onDelete: Cascade)

    issues    Issue

    createdAt DateTime @default(now())

    @@index([teamId, startsAt])
    @@map("cycles")
}

// Комментарий
model Comment {
    id    String @id @default(cuid())
    content String @db.Text
    issueId Int
    issue    Issue @relation(fields: [issueId], references: [id], onDelete: Cascade)
    authorId String
    author    User @relation(fields: [authorId], references: [id])

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    @@map("comments")
}

```

```
}
```

```
// Теги (Labels)
```

```
model Label {  
  id    String    @id @default(cuid())  
  name  String  
  color String     // Hex code  
  teamId String  
  team  Team       @relation(fields: [teamId], references: [id], onDelete: Cascade)  
  issues LabelOnIssue  
  
  @@unique([teamId, name])  
  @@map("labels")  
}
```

```
// Таблица связи многие-ко-многим для Label и Issue
```

```
model LabelOnIssue {  
  issueId Int  
  labelId String  
  issue  Issue @relation(fields: [issueId], references: [id], onDelete: Cascade)  
  label  Label @relation(fields: [labelId], references: [id], onDelete: Cascade)  
  
  @@id([issueId, labelId])  
  @@map("_LabelToIssue")  
}
```

```
// История изменений задачи (Audit Log)
```

```
model IssueHistory {  
  id    String @id @default(cuid())  
  issueId Int  
  issue  Issue @relation(fields: [issueId], references: [id], onDelete: Cascade)  
  actorId String  
  actor  User  @relation(fields: [actorId], references: [id])  
  field  String // Какое поле изменилось (напр. "status", "priority")  
  oldValue String?  
  newValue String?  
  
  createdAt DateTime @default(now())  
  
  @@map("issue_history")  
}
```

4.3. Стратегия индексации

Для обеспечения высокой производительности запросов (особенно для дашбордов и Kanban-досок) необходимо создать правильные индексы:

- `@@index([teamId, statusId])`: Критичен для рендеринга Kanban-доски, где мы выбираем все задачи команды, сгруппированные по статусам.
- `@@index([cycleId])`: Используется для быстрого отображения задач текущего спринта.
- `@@index([assigneeId])`: Необходим для страницы "My Issues".
- Полнотекстовый поиск (FTS) будет реализован средствами PostgreSQL (GIN index на поля `title` и `description`) для поддержки быстрого поиска через Command Palette.

5. Система безопасности и контроль доступа (RBAC)

Безопасность в BugZero строится на принципе "Defense in Depth" (глубокоэшелонированная защита). Основной уровень защиты — это Row Level Security (RLS) в базе данных.

5.1. Ролевая модель (RBAC Matrix)

Определены три основные роли, которые хранятся в таблице `TeamMember`:

Действие / Роль	GUEST (Гость)	MEMBER (Участник)	ADMIN (Админ)
Просмотр задач	Только свои или назначенные	Все задачи команды	Все задачи команды
Создание задач	Да	Да	Да
Редактирование (Описание, Статус)	Только свои задачи	Любые задачи	Любые задачи
Удаление задач	Нет	Нет	Да
Управление спринтами (Создание/Закрыт	Нет	Да	Да

ие)			
Управление настройками команды	Нет	Нет	Да
Приглашение пользователей	Нет	Нет	Да

5.2. Реализация RLS Политик (Supabase SQL)

Политики RLS гарантируют, что SQL-запросы будут возвращать только разрешенные строки, независимо от того, как они были вызваны (через API или напрямую). Ниже приведены конкретные примеры политик для таблицы issues.²⁵

SQL

-- Включение RLS

```
ALTER TABLE issues ENABLE ROW LEVEL SECURITY;
```

-- 1. Политика на ЧТЕНИЕ (SELECT)

-- Пользователь может видеть задачу, если он является участником команды, к которой задача принадлежит.

```
CREATE POLICY "View issues based on team membership"
```

```
ON issues FOR SELECT
```

```
USING (
```

```
  EXISTS (
```

```
    SELECT 1 FROM team_members
```

```
    WHERE team_members.team_id = issues.team_id
```

```
    AND team_members.user_id = auth.uid()
```

```
  )
```

```
);
```

-- 2. Политика на ВСТАВКУ (INSERT)

-- Пользователь может создавать задачи только в тех командах, где он является участником.

```
CREATE POLICY "Create issues if team member"
```

```
ON issues FOR INSERT
```

```
WITH CHECK (
```

```

EXISTS (
  SELECT 1 FROM team_members
  WHERE team_members.team_id = issues.team_id
  AND team_members.user_id = auth.uid()
)
);

```

-- 3. Политика на ОБНОВЛЕНИЕ (UPDATE)

-- Участники команды могут обновлять задачи.

-- Дополнительно можно добавить логику, что GUEST может обновлять только назначенные на него задачи.

CREATE POLICY "Update issues if team member"

ON issues FOR UPDATE

USING (

```

  EXISTS (
    SELECT 1 FROM team_members
    WHERE team_members.team_id = issues.team_id
    AND team_members.user_id = auth.uid()
  )
);

```

-- 4. Политика на УДАЛЕНИЕ (DELETE)

-- Только администраторы команды могут удалять задачи.

CREATE POLICY "Admins can delete issues"

ON issues FOR DELETE

USING (

```

  EXISTS (
    SELECT 1 FROM team_members
    WHERE team_members.team_id = issues.team_id
    AND team_members.user_id = auth.uid()
    AND team_members.role = 'ADMIN'
  )
);

```

Важное замечание по безопасности: Использование `auth.uid()` в политиках Supabase позволяет безопасно идентифицировать пользователя. Для оптимизации производительности запросов с RLS, столбцы `team_id` и `user_id` в таблице `team_members` должны быть проиндексированы, так как они участвуют в каждом запросе проверки прав.³⁶

6. API и серверная логика

6.1. Server Actions vs API Routes

В BugZero мы отказываемся от классического REST API для внутреннего взаимодействия фронтенда и бэкенда в пользу **Server Actions**.

- **Server Actions (Мутации):** Все действия пользователя (создание задачи, изменение приоритета) обрабатываются серверными действиями. Это упрощает код: не нужно создавать отдельные эндпоинты, сериализовать JSON и обрабатывать HTTP-статусы вручную. Функция на сервере вызывается как обычная JS-функция.
 - *Пример:* `updateIssueStatus(issueId, newStatus)` — это асинхронная серверная функция, которую мы импортируем прямо в клиентский компонент кнопки.
- **API Routes (Интеграции):** Классические API Routes (`app/api/webhooks/github/route.ts`) будут использоваться исключительно для внешних интеграций, например, для приема вебхуков от GitHub при пуше коммитов или мердж-реквестах.¹⁸

6.2. Вебхуки и интеграции

Для реализации требования "функциональности" система должна уметь общаться с внешним миром.

- **GitHub Integration:** При получении вебхука от GitHub, система должна парсить сообщение коммита. Если в нем есть текст `Fixes BACK-123`, система автоматически переводит задачу `BACK-123` в статус `DONE` или `IN REVIEW`.
- **Slack Notifications:** Отправка уведомлений в канал команды при создании критических багов (`Priority = URGENT`).

6.3. Rate Limiting и валидация

Даже при использовании Server Actions необходима защита от злоупотреблений.

- **Валидация данных:** Использование библиотеки `Zod` для строгой валидации всех входных данных на сервере перед записью в БД.
- **Rate Limiting:** Ограничение количества запросов на создание задач от одного пользователя (например, не более 10 задач в минуту) для защиты от спама, реализованное через `Redis` или встроенные средства платформы (Upstash).³⁷

7. Пользовательский интерфейс (UI) и UX спецификации

Интерфейс BugZero должен быть аскетичным, но мощным. Основной упор делается на типографику, отступы и скорость реакции.

7.1. Command Palette (Командная палитра)

Это центральный элемент навигации и управления для power-users.

- **Вызов:** Глобальный шорткат `Cmd+K` (macOS) / `Ctrl+K` (Windows).

- **Технология:** Реализация на базе cmdk (Radix UI) для доступности и производительности.³⁸
- **Контекстные команды:** Меню должно адаптироваться к текущей странице.
 - В глобальном контексте: "Go to Project...", "Create Issue", "Switch Team", "My Settings".
 - В контексте задачи: "Change Status", "Assign to...", "Set Priority", "Copy Branch Name".
 - В контексте доски: "Filter by Assignee", "Toggle Dark Mode".
- **Нечеткий поиск (Fuzzy Search):** Алгоритм поиска должен прощать опечатки и понимать сокращения (например, ввод "crt iss" должен предлагать "Create Issue").

7.2. Оптимистичный UI (Optimistic Updates)

Для обеспечения метрики "ощущается мгновенным", мы используем хук useOptimistic из React 18/19.³⁰

Сценарий изменения статуса задачи:

1. Пользователь перетаскивает карточку из "Todo" в "Done".
2. UI *мгновенно* перерисовывает карточку в новой колонке, обновляя локальный стейт через useOptimistic.
3. Параллельно отправляется асинхронный Server Action updateIssueStatus в базу данных.
4. Если сервер возвращает успех — UI синхронизируется с реальными данными (обычно незаметно для пользователя).
5. Если сервер возвращает ошибку — UI автоматически "отскакивает" (rollback) назад, и пользователю показывается Toast-уведомление с ошибкой.

7.3. Организация представлений (Views)

- **List View (Список):** Основной вид для работы с бэклогом. Должен поддерживать:
 - Изменение ширины колонок.
 - Сортировку по любому полю.
 - Мульти-выделение (Shift+Click) для массового редактирования (Bulk Actions).
- **Board View (Kanban):**
 - Горизонтальный скролл.
 - Группировка (Swimlanes) по приоритетам или исполнителям.
 - Drag-and-drop с использованием библиотеки dnd-kit или @hello-pangea/dnd.

8. Нефункциональные требования (NFR) и SLA

Для обеспечения качества уровня Enterprise SaaS, система должна соответствовать строгим метрикам.¹⁴

8.1. Производительность (Performance Metrics)

- **LCP (Largest Contentful Paint):** < 1.2 сек. Основной контент (список задач) должен быть виден почти мгновенно.
- **FID (First Input Delay):** < 100 мс. Интерфейс должен быть отзывчивым сразу после загрузки.
- **API Response Time:** 95% запросов (p95) должны выполняться быстрее 200 мс.
- **Optimistic UI Latency:** Визуальный отклик на действие пользователя < 50 мс.

8.2. Надежность и доступность (Reliability)

- **Availability (Uptime):** 99.9% (допустимый простой не более 43 минут в месяц).
- **RPO (Recovery Point Objective):** < 1 часа. Потеря данных в случае катастрофического сбоя не должна превышать 1 часа (обеспечивается WAL-логами PostgreSQL и Point-in-Time Recovery в Supabase).
- **RTO (Recovery Time Objective):** < 4 часов на полное восстановление работоспособности.

8.3. Масштабируемость (Scalability)

- Система должна поддерживать работу с таблицами до 1 млн записей (Issues) без деградации производительности выборки (благодаря индексам и пагинации).
- Поддержка до 1000 одновременных подключений через WebSocket (Realtime) на один инстанс.

9. План реализации (Roadmap) и стратегия тестирования

9.1. Этапы разработки

1. **Фаза 1: Core & Schema (Недели 1-2):** Настройка репозитория, Supabase, Prisma. Реализация схемы БД и миграций. Настройка аутентификации.
2. **Фаза 2: Basic CRUD & Issues (Недели 3-4):** Реализация создания, просмотра и редактирования задач. Server Actions. RLS политики.
3. **Фаза 3: UI/UX & Command Palette (Недели 5-6):** Внедрение cmdk, оптимистичных обновлений, Drag-and-drop доски.
4. **Фаза 4: Realtime & Collaboration (Недели 7-8):** Подключение WebSocket, комментариев, уведомлений.
5. **Фаза 5: Polish & Launch (Недели 9+):** Тестирование, исправление багов, деплой.

9.2. Стратегия тестирования

- **Unit Testing:** Тестирование утилитных функций и валидаторов (Vitest).
- **Integration Testing:** Тестирование Server Actions и взаимодействия с БД (используя

тестовую БД в Docker).

- **E2E Testing:** Сквозное тестирование критических путей (Login -> Create Issue -> Move to Done) с использованием **Playwright**. Это гарантирует, что основной функционал работает с точки зрения пользователя.⁴¹

Данное техническое задание описывает создание современной, быстрой и надежной системы, которая решает проблемы классических трекеров за счет использования передовых архитектурных паттернов и технологий 2025 года.

Works cited

1. Best 12 Issue and Project Tracking Software - Lark, accessed on January 22, 2026, https://www.larksuite.com/en_us/blog/issue-and-project-tracking-software
2. Best 12 Issue Tracking Software Solutions in 2025 - AI4Dev, accessed on January 22, 2026, <https://ai-4.dev/software-testing/best-issue-tracking-software/>
3. Our journey to local-first - HEY World, accessed on January 22, 2026, <https://world.hey.com/zac.wood/our-journey-to-local-first-2ade26a5>
4. Optimistic UI with Server Actions in Next.js: A Smoother User Experience - Medium, accessed on January 22, 2026, <https://medium.com/@mishal.s.suyog/optimistic-ui-with-server-actions-in-next-js-a-smoother-user-experience-6b779e4293a9>
5. Issue tracking software comparison: The top 20 tools for development teams in 2026, accessed on January 22, 2026, <https://monday.com/blog/rnd/issue-tracker/>
6. Command Palette UI Design: Best practices, Design variants & Examples - Mobbin, accessed on January 22, 2026, <https://mobbin.com/glossary/command-palette>
7. Bug tracking software: essential tools for modern development teams - Monday.com, accessed on January 22, 2026, <https://monday.com/blog/rnd/bug-tracking-software/>
8. Sprints with Kanban: How to Structure Your Scrum Process with Kanban - Businessmap, accessed on January 22, 2026, <https://businessmap.io/blog/kanban-sprint>
9. The Best Linear Alternatives for GitHub-First Teams | Zenhub Blog, accessed on January 22, 2026, <https://www.zenhub.com/blog-posts/the-best-linear-alternatives-for-github-first-teams>
10. Bug Tracking: A Complete Guide for Developers and Testers - LambdaTest, accessed on January 22, 2026, <https://www.testmu.ai/learning-hub/bug-tracking/>
11. The Ultimate Guide to Bug Tracking: Strategies, Tools, and Best Practices - BugHerd, accessed on January 22, 2026, <https://bugherd.com/article/the-ultimate-guide-to-bug-tracking-strategies-tools-and-best-practices>
12. Issue Tracking Tools Review 2019 by JetBrains, accessed on January 22, 2026, <https://www.jetbrains.com/lp/tracking-tools-review-2019/>
13. GDPR Compliance Checklist: Complete 2025 Guide for B2B SaaS Companies -

- ComplyDog, accessed on January 22, 2026,
<https://complydog.com/blog/gdpr-compliance-checklist-complete-guide-b2b-saas-companies>
14. Types of Service Level Agreement (SLA) Metrics - IBM, accessed on January 22, 2026, <https://www.ibm.com/think/topics/sla-metrics>
 15. Service Level Agreement Metrics: Benchmarking SLAs with Contract Intelligence, accessed on January 22, 2026,
<https://blog.term scout.com/service-level-agreement-metrics-benchmarking-fair-performance-standards>
 16. Next.js 16 vs Remix vs Astro: Choosing the Right React Framework in 2025, accessed on January 22, 2026,
<https://dev.to/saswatapal/nextjs-16-vs-remix-vs-astro-choosing-the-right-react-framework-in-2025-3lio>
 17. Next.js Server Actions vs API Routes: Don't Build Your App Until You Read This, accessed on January 22, 2026,
<https://dev.to/myogeshchavan97/nextjs-server-actions-vs-api-routes-dont-build-your-app-until-you-read-this-4kb9>
 18. A Deep Dive into Next.js Server Actions and API Routes - CloudThat, accessed on January 22, 2026,
<https://www.cloudthat.com/resources/blog/a-deep-dive-into-next-js-server-actions-and-api-routes>
 19. Next.js vs Remix 2025: Complete Developer Framework Guide - Strapi, accessed on January 22, 2026,
<https://strapi.io/blog/next-js-vs-remix-2025-developer-framework-comparison-guide>
 20. React vs Vue: which one should you choose in 2025? - DECODE agency, accessed on January 22, 2026, <https://decode.agency/article/react-vs-vue/>
 21. Real-Time Collaboration Tools: Supabase vs. Firebase - Propelius Technologies, accessed on January 22, 2026,
<https://propelius.ai/blogs/real-time-collaboration-tools-supabase-vs-firebase>
 22. Supabase vs Firebase in 2025: The Ultimate BaaS Comparison - AnotherWrapper, accessed on January 22, 2026,
<https://anotherwrapper.com/blog/supabase-vs-firebase>
 23. Best database platform for realtime updates? (Supabase, Firebase, etc) - Reddit, accessed on January 22, 2026,
https://www.reddit.com/r/reactnative/comments/1pbpr14/best_database_platform_for_realtime_updates/
 24. Supabase or firebase? Which do you prefer? : r/FlutterDev - Reddit, accessed on January 22, 2026,
https://www.reddit.com/r/FlutterDev/comments/1lsdgyn/supabase_or_firebase_which_do_you_prefer/
 25. Lock Down Your Data: Implement Row-Level Security Policies in Supabase SQL, accessed on January 22, 2026,
<https://dev.to/thebenforce/lock-down-your-data-implement-row-level-security-policies-in-supabase-sql-4p82>

26. Authorization via Row Level Security | Supabase Features, accessed on January 22, 2026, <https://supabase.com/features/row-level-security>
27. How to use Prisma ORM with multiple database schemas, accessed on January 22, 2026, <https://www.prisma.io/docs/orm/prisma-schema/data-model/multi-schema>
28. Connecting Prisma to Supabase in Next.js | by Unalysuf - Medium, accessed on January 22, 2026, <https://medium.com/@unalysuf/connecting-prisma-to-supabase-in-next-js-20a7ca546ddb>
29. Prototyping your schema | Prisma Documentation, accessed on January 22, 2026, <https://www.prisma.io/docs/orm/prisma-migrate/workflows/prototyping-your-schema>
30. Understanding optimistic UI and React's useOptimistic Hook - LogRocket Blog, accessed on January 22, 2026, <https://blog.logrocket.com/understanding-optimistic-ui-react-useoptimistic-hook/>
31. How to Use the Optimistic UI Pattern with the useOptimistic() Hook in React - freeCodeCamp, accessed on January 22, 2026, <https://www.freecodecamp.org/news/how-to-use-the-optimistic-ui-pattern-with-the-useoptimistic-hook-in-react/>
32. schema.prisma - finom/realtime-kanban - GitHub, accessed on January 22, 2026, <https://github.com/finom/realtime-kanban/blob/main/prisma/schema.prisma>
33. Database schema - Developer, Atlassian, accessed on January 22, 2026, <https://developer.atlassian.com/server/jira/platform/database-schema/>
34. Row Level Security | Supabase Docs, accessed on January 22, 2026, <https://supabase.com/docs/guides/database/postgres/row-level-security>
35. Supabase Row Level Security Explained With Real Examples | by debug_senpai - Medium, accessed on January 22, 2026, <https://medium.com/@jigsz6391/supabase-row-level-security-explained-with-real-examples-6d06ce8d221c>
36. How to Manage Row-Level Security Policies Effectively in Supabase - Medium, accessed on January 22, 2026, <https://medium.com/@jay.digitalmarketing09/how-to-manage-row-level-security-policies-effectively-in-supabase-98c9dfbc2c01>
37. Securing your API | Supabase Docs, accessed on January 22, 2026, <https://supabase.com/docs/guides/api/securing-your-api>
38. Command Component - Modern UI, accessed on January 22, 2026, <https://modern-ui.org/docs/components/command>
39. Nonfunctional Requirements Explained: Examples, Types, Tools, accessed on January 22, 2026, <https://www.modernrequirements.com/blogs/what-are-non-functional-requirements/>
40. Evaluating Service Level Agreements: The 8 Best SLA Metrics - top.legal, accessed on January 22, 2026, <https://www.top.legal/en/knowledge/sla-metrics>
41. React Native Tech Stack: Build High-Performance Mobile Apps Efficiently | FAB

Web Studio, accessed on January 22, 2026,

<https://fabwebstudio.com/blog/react-native-tech-stack-build-high-performance-mobile-apps-efficiently>